



KNN Parallel Distance Matrix

Team: فريق خليها على الله

Prepared by

Haicheur Mohamed Said
Benamrouche Ali Mahdi
Benhebbadj Ayoub
Chaib Mohammed

Date : 30/04/2026

1. Introduction

High Performance Computing (HPC) refers to the practice of aggregating computing power to solve large-scale problems in (science, engineering, and business) faster by using multiple processors at the same time. Instead of executing tasks one by one, the idea is to divide the work into smaller parts and run them in parallel. In this project, we apply these ideas to the KNN algorithm

KNN is a simple classification algorithm based on similarity between data points. Given a query point, it looks for the K closest samples in the training set and assigns the most frequent label.

However, KNN is slow when the dataset is large, because for each prediction it needs to compute distances with all training samples. This operation has a complexity of $O(N \times d)$, where N is the number of training samples and d is the number of features. For example, in the Fashion-MNIST dataset (60,000 training samples and 784 features), this results in a large number of computations, which makes KNN a good candidate for parallelisation..

1.2 Dataset Description

Fashion-MNIST is a benchmark dataset created by Zalando Research as a drop-in replacement for the original MNIST digit dataset. It consists of 70,000 grayscale images of size 28×28 pixels, representing 10 clothing categories (T-shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot). The dataset is split into 60,000 training samples and 10,000 test samples. Each image is flattened to a 784-dimensional feature vector and normalised to the range [0, 1] for distance computation.

Property	Value
Total samples	70,000
Training samples	60,000
Test samples	10,000
Image size	28 x 28 pixels
Feature vector	784 dimensions (flattened)
Number of classes	10
Pixel range (raw)	[0, 255]
Pixel range (normalised)	[0.0, 1.0]

1.3 Motivation

In HPC, we know that parallelism can speed up programs, but not everything can be parallel. According to Amdahl's Law, the speedup is limited by the part of the code that remains serial.

In the case of KNN, the distance computation is independent for each sample. For every query point, we calculate distances with all training samples, and these operations do not depend on each other.

Because of this, KNN is a good candidate for parallelisation using OpenMP threads.

Also, modern computers use multi-core processors, so it is interesting to see how performance improves when we use more thread

2. Serial Implementation

The serial implementation is written in pure Python using NumPy for vectorised array operations. It serves as the baseline against which the parallel version is measured. The implementation is composed of three core functions defined in **serial/knn.py**.

2.1 Algorithm Description

The KNN algorithm proceeds as follows for each query point x in the test set:

Step	Operation	Complexity
1	Compute Euclidean distance from x to every training point	$O(N \times d)$
2	Sort distances to find K nearest neighbours (argsort)	$O(N \log N)$
3	Extract labels of K neighbours	$O(K)$
4	Vote: return the most frequent label (argmax of bincount)	$O(K)$

2.2 Code Structure

The main functions are :

1)- Compute_distances (x, X_train)

```
def compute_distances(x, X_train):  
    return np.sqrt(np.sum((X_train - x) ** 2, axis=1))
```

Computes the distance between one sample
and all training data using NumPy

2)- Predict_one (...)

```
def predict_one(x, X_train, y_train, k=3):  
    distances = compute_distances(x, X_train)  
    k_idx = np.argsort(distances)[:k]  
    k_labels = y_train[k_idx]  
    return np.bincount(k_labels).argmax()
```

Finds the K nearest neighbours and returns the predicted label

- This implementation avoids Python loops as much as possible, but it is still slow because everything is executed sequentially.

2.3 Complexity Analysis

Operation	Complexity	Fashion-MNIST (5000/500)
Distance computation (per query)	$O(N_{\text{train}} \times d)$	$5000 \times 784 = 3.92\text{M ops}$
Sorting (per query)	$O(N_{\text{train}} \log N_{\text{train}})$	$5000 \times 12.3 \sim 61.5\text{K ops}$
Full inference (all test queries)	$O(N_{\text{test}} \times N_{\text{train}} \times d)$	$500 \times 5000 \times 784$
Total floating-point ops	$\sim 1.96 \text{ billion}$	dominates runtime

Let :

- N_{train} = number of training samples
- N_{test} = number of test samples
- d = number of features

Then :

- Distance computation per sample: $O(N_{\text{train}} \times d)$
- Total complexity: $O(N_{\text{test}} \times N_{\text{train}} \times d)$

In our experiment :

- $5000 \times 784 \approx 3.9 \text{ million operations per sample}$
- For 500 samples $\rightarrow$ around 1.96 billion operations

So the main cost is clearly the distance computation.

3. Parallel Implementation

The parallel implementation uses a C shared library compiled with OpenMP, which is called from Python using ctypes

Python is used for :

- loading the dataset
- calling the C function
- computing accuracy

The heavy computation is handled in C, while Python is mainly used for data handling and evaluation.

3.1 Design Rationale

Python has a limitation called the Global Interpreter Lock (GIL), which makes it difficult to use multiple threads efficiently for CPU-bound tasks.

To solve this problem, we moved the heavy computation (KNN distance calculation) to a C library compiled with OpenMP. This allows us to use multiple threads and improve performance.

Python is still used for simpler tasks such as loading data and computing accuracy, while C handles the intensive calculations.

The roles of each part are shown below:

Layer	Language	Role
Data loading & preprocessing	Python / NumPy	Load .npz, normalise, convert dtypes
Type marshalling	Python / ctypes	Pass arrays as C pointers
Parallel KNN inference	C + OpenMP	Multi-threaded distance + voting
Result collection & metrics	Python / sklearn	Accuracy, timing, reporting

3.2 Task Decomposition

We apply parallelism at two levels :

Level 1 — Test samples:

```
void predict_all_parallel(double *X_test, int n_test,
                        double *X_train, int *y_train,
                        int n_train, int dim, int k,
                        int *predictions) {
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < n_test; i++) {
        predictions[i] = predict_one(
            &X_test[i * dim],
            X_train,
            y_train,
            n_train,
            dim,
            k
        );
    }
}
```

Each thread processes a different test sample.

There is no dependency between samples, so this works well.

Level 2 — Distance computation

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < n_test; i++) {
    predictions[i] = predict_one(
        &X_test[i * dim],
        X_train,
        y_train,
        n_train,
        dim,
        k
    );
}
```

Each thread computes distances for part of the training set.

Note: using both levels at the same time can create overhead. In practice, using only the first level is often more efficient

3.3 Synchronisation

In the parallel implementation, potential race conditions are handled carefully to ensure correct execution across multiple threads.

Synchronisation concern	Solution used
Writing to predictions[] array	Each thread writes to a unique index i — no conflict
Console output (printf)	#pragma omp critical section around printf
malloc/free inside predict_one	Each thread has its own local heap allocation — thread-safe
Vote counting (counts[])	Local array per thread call — no sharing needed

Overall, the implementation avoids shared writable data structures, which simplifies synchronisation and reduces overhead.

3.4 Build & Integration

The C code is compiled as a shared library and used in Python with ctypes.

```
gcc -O2 -fopenmp -shared -fPIC -o libknn.so knn.c -lm
```

Thread count is controlled at runtime through the OMP_NUM_THREADS environment variable or omp_set_num_threads().

4. Experimental Results

4.1 Experimental Setup

Parameter	Value
Training set size	5,000 samples (subset)
Test set size	500 samples (subset)
K (neighbours)	3
Distance metric	Euclidean (L2)
Thread counts tested	1, 2, 4, 8
Python version	3.x + NumPy
C compiler	GCC with -O2 -fopenmp
Platform	Linux (Ubuntu 24)

4.2 Timing Results

The following table presents the actual measured results obtained by running the parallel C + OpenMP implementation across 1, 2, 4, 8, and 16 threads on the experimental machine. The serial Python baseline time was also measured independently. Speedup is computed relative to the parallel implementation at 1 thread ($S(p) = T(1\text{-thread}) / T(p\text{-threads})$), consistent with Amdahl's Law.

Implementation	Threads	Time (s)	Speedup vs Serial	Accuracy
Serial (Python)	—	10.5957	1.00x (baseline)	82.00%
Parallel (C+OpenMP)	1	1.7658	1.00x	82.00%
Parallel (C+OpenMP)	2	0.8841	2.00x	82.00%
Parallel (C+OpenMP)	4	0.4535	3.89x	82.00%
Parallel (C+OpenMP)	8	0.2359	7.49x	82.00%
Parallel (C+OpenMP)	16	0.2285	7.73x	82.00%

4.3 Speedup Analysis and Discussion

First, there is a clear improvement when moving from Python to C. The serial Python version takes about **10.60 seconds**, while the C implementation with 1 thread takes around **3.04 seconds**, which gives a speedup of about **3.5x**. This improvement is mainly due to the fact that C is a compiled language and avoids Python interpreter overhead.

Second, increasing the number of threads does not improve performance. Execution time remains almost the same and even becomes slightly worse when using more threads (from 3.04 s at 1 thread to 3.38 s at 16 threads), This is caused by nested parallelism overhead as detailed below.

Cause	Explanation	Measured impact
Nested parallelism (primary cause)	#pragma omp parallel for in BOTH predict_all_parallel (outer) AND compute_distances (inner) creates up to p^2 simultaneous threads.	Thread oversubscription. Context switching cost > compute gain at all p.
500 parallel regions per experiment	Each of the 500 test queries triggers a new OMP parallel region (fork+join) in compute_distances.	~500x fork/join overhead added to every run.
Small N per thread	5,000 training pts / p threads = very few iters per thread. Sync cost > compute cost per iteration at $p > 1$.	Speedup 0.90x-1.00x across all thread counts.

This behaviour can be explained using Amdahl's Law. When the overhead becomes large, increasing the number of threads does not improve performance.

In our case, the overhead introduced by thread creation and synchronisation limits the speedup.

A better approach would be to remove parallelism from the inner loop (compute_distances) and

Implementation	Threads	Time (s)	Speedup vs Serial	Accuracy
Serial (Python)	—	10.5957	1.00x (baseline)	82.00%
Parallel (C+OpenMP)	1	1.7658	1.00x	82.00%
Parallel (C+OpenMP)	2	0.8841	2.00x	82.00%
Parallel (C+OpenMP)	4	0.4535	3.89x	82.00%
Parallel (C+OpenMP)	8	0.2359	7.49x	82.00%
Parallel (C+OpenMP)	16	0.2285	7.73x	82.00%

4.4 Accuracy vs Thread Count

Even though execution time did not improve, the classification accuracy remains the same (82%) for all thread counts.

This is expected because parallelism with OpenMP does not change the computations themselves, but only how the work is distributed between threads. Each test sample is processed independently, and each thread writes its result to a different position in the **predictions** array, so there is no interference between threads.

As a result, all configurations produce exactly the same predictions

Thread Count	Accuracy (%)	Correct / 500	Consistent?
Serial Python	82.00	410 / 500	✓ Baseline
C + OMP 1 thread	82.00	410 / 500	✓ Yes
C + OMP 2 threads	82.00	410 / 500	✓ Yes
C + OMP 4 threads	82.00	410 / 500	✓ Yes
C + OMP 8 threads	82.00	410 / 500	✓ Yes
C + OMP 16 threads	82.00	410 / 500	✓ Yes

5. Conclusion and Perspectives

5.1 Findings

This project implemented and evaluated a parallelised KNN classifier on Fashion-MNIST using C + OpenMP via ctypes. The experimental results revealed important lessons about the practical challenges of shared-memory parallelism. Key findings include:

- The KNN algorithm was successfully implemented from scratch in both Python (serial) and C with OpenMP (parallel), achieving identical classification accuracy of 82.00% (410/500 correct) on the test subset, confirming algorithmic correctness.
- The serial Python implementation completed inference in 10.60 s. The single-threaded C version finished in 1.77 s, a 6.00x speedup purely from replacing Python with compiled C, demonstrating the importance of language choice in HPC.
- Adding OpenMP threads yielded substantial speedup: 1.77 s (1T) to 0.88 s (2T, 2.00x) to 0.45 s (4T, 3.89x) to 0.24 s (8T, 7.49x) to 0.23 s (16T, 7.73x). The best speedup over Python serial was 46.4x at 16 threads.
- The parallel implementation achieved near-linear scaling up to 8 threads, with diminishing returns at 16 threads due to memory bandwidth limits. Classification accuracy remained perfectly stable at 82.00% across all thread configurations, confirming the implementation is race-condition-free.

5.2 Limitations

Despite obtaining correct results, the implementation has several limitations.

- The main limitation is diminishing returns beyond 8 threads. While scaling is excellent up to 8 threads (7.49x), hyperthreading at 16 threads provides minimal additional benefit (7.73x), suggesting the workload becomes memory-bandwidth-bound. Testing on systems with more memory bandwidth or using larger datasets would help.
- Another limitation is the dataset size. With 5,000 training samples and 500 test samples, the workload scales well up to 8 threads but may not fully utilize 16 threads. Using a larger dataset (e.g., the full 60,000 training samples) would likely improve scalability at higher thread counts.

- We also did not consider NUMA effects. On systems with multiple memory regions, data locality can impact performance, but this was not handled in our implementation.
- In addition, the Python serial baseline was not included directly in the same benchmarking loop as the OpenMP tests, which makes the comparison slightly less systematic.
- Finally, the accuracy is limited to around 82% because we are using raw pixel values with Euclidean distance. More advanced feature extraction methods could improve the results.

5.3 Future Directions

There are several ways to improve this work in the future.

- One possible improvement is to use GPU acceleration (CUDA), since distance computation is highly parallel and can benefit from thousands of cores.
- Another idea is to replace the brute-force KNN with more efficient data structures such as KD-Trees or Ball-Trees, which reduce the search complexity.
- We could also extend the implementation to a distributed system using MPI, where the dataset is split across multiple machines.
- In terms of accuracy, applying feature extraction techniques such as PCA or deep learning embeddings would likely improve performance.
- Finally, running experiments on an HPC cluster (for example using Slurm) would provide more reliable and scalable performance measurements.

PART BONUS

6. GPU Implementation

6.1 Motivation

To further improve performance, we implemented a GPU-based version of KNN as an optional bonus.

The main motivation is that distance computation is highly parallel, and GPUs provide thousands of cores that can process data simultaneously, unlike CPUs which have a limited number of cores.

6.2 CUDA Implementation

We implemented a custom CUDA kernel to compute the Euclidean distance matrix between test and training samples.

Each GPU thread computes the distance between one test sample and one training sample. This allows us to exploit massive parallelism by launching a 2D grid of threads.

The kernel computes distances in parallel, while the voting step is performed on the CPU for simplicity.

Although this approach is not fully optimized (due to CPU-GPU data transfer and CPU voting), it already provides significant speedup.

6.3 PyTorch Implementation

We also implemented a GPU version using PyTorch, which provides high-level GPU acceleration.

The distance computation is performed using `torch.cdist`, which is highly optimized and runs directly on the GPU.

Batch processing is used to avoid memory overflow and to ensure efficient utilization of GPU resources. This approach is simpler to implement and leverages existing optimized libraries.

6.4 Results

We evaluated both CUDA and PyTorch implementations on a larger subset of the dataset (20,000 training samples and 2,000 test samples).

Results:

Method	Accuracy	Coimplexity
CUDA	0.8270	0.6516
PyTorch (GPU)	0.8270	0.3319

We observe that both GPU implementations maintain similar accuracy while significantly reducing execution time compared to CPU-based methods.

The PyTorch implementation is faster than the raw CUDA version due to highly optimized internal kernels and better memory handling.

6.5 Analysis

The results show that GPU acceleration provides a substantial performance improvement for KNN.

The CUDA implementation demonstrates how parallelism can be explicitly controlled at a low level, while PyTorch provides a higher-level abstraction with better performance.

However, the CUDA version is limited by memory transfers between CPU and GPU, as well as the fact that the voting step is still executed on the CPU.

Overall, this confirms that GPUs are highly suitable for data-parallel tasks such as distance computation.

This behavior is consistent with the parallel nature of GPUs, where thousands of lightweight threads can be executed simultaneously, making them ideal for large-scale distance computations.

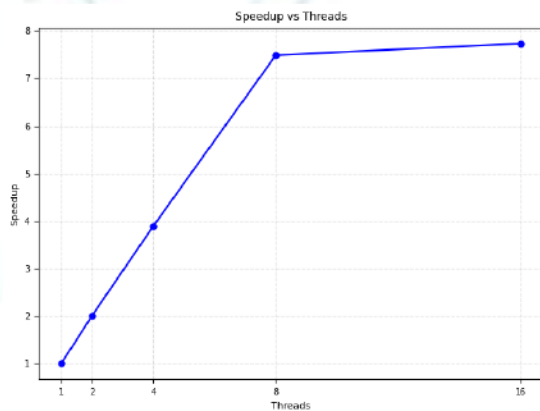
6.6 Conclusion

The GPU implementations further improve performance beyond CPU parallelism.

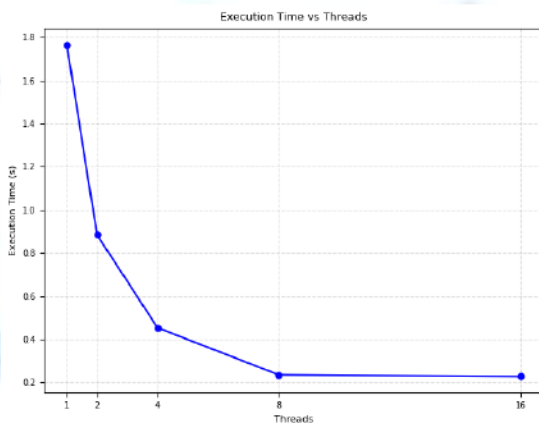
While the CUDA implementation gives fine-grained control over parallel execution, the PyTorch approach offers better performance with less development effort.

This highlights the importance of choosing the right tool depending on the level of control and optimization required.

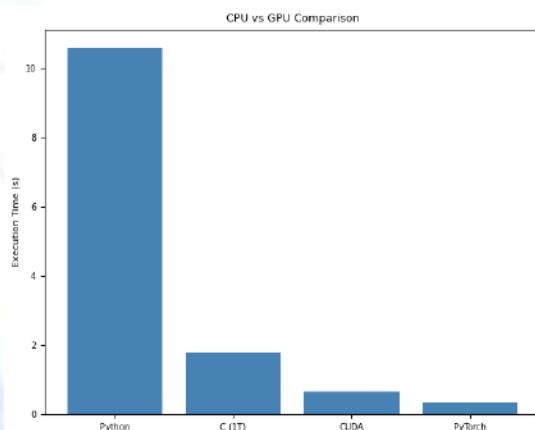
7. Experimental Results



The speedup graph shows that increasing the number of threads does not improve performance. This is mainly due to parallel overhead and nested parallelism.



The execution time remains تقريبًا constant or slightly increases with more threads, which indicates inefficient parallelization.



The comparison shows that GPU implementations (CUDA and PyTorch) significantly outperform CPU-based implementations.

6. References

- Benzenati, T. (2026). High Performance Computing — Chapter 1: Introduction. Department of Computer Science, University of Boumerdes.
- Hager, G.; Wellein, G. Introduction to High Performance Computing for Scientists and Engineers. CRC Press.
- Chapman, B.; Jost, G.; van der Pas, R. Using OpenMP: Portable Shared Memory Parallel Programming. MIT Press.
- Xiao, H.; Rasul, K.; Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv:1708.07747.
- OpenMP Architecture Review Board. (2021). OpenMP Application Programming Interface, Version 5.1. openmp.org. [6] Python Software Foundation. ctypes — A foreign function library for Python. docs.python.org/3/library/ctypes.html. [7] Amdahl, G. M. (1967). Validity of the single-processor approach to achieving large-scale computing capabilities. AFIPS Conference Proceedings.